Sublementary Material for Paper: Fuzzy Private Set Intersection with Large Databases via Locality-Sensitive Hashing

A ADDITIONAL CONTENT ON THE FUZZY PROBABILISTIC DATASTRUCTURE

A.1 Proof of Theorem 3.1

We first start by enumerating our assumptions.

(1) **Independence of LSH evaluations.** The hash functions $\{H_{\ell,k}\}_{\ell\in[L],\,k\in[K]}$ are sampled independently from the LSH family $\mathcal{H}$. In particular, for any fixed $q^*, y^* \in \mathcal{U}$, the events
$$H_{\ell,k}(q^*) = H_{\ell,k}(y^*)$$
are independent over all $(\ell, k) \in [L] \times [K]$.
(2) **Successful setup and insertion.** All insertions into the Fuzzy PDS succeed. Equivalently, we condition on the event that the underlying PDS does not overflow or abort during setup or the insertion of Y.
(3) **AND-OR semantics across lookups.** For a fixed hash instance $\ell \in [L]$, we compute K LSH values, and concatenate them and hash the result (GetFuzzyVals). A PRF is applied to each element of the resulting vector, the L corresponding PDS addresses are obtained by running GetInfo, and then finally a PDS lookup using the result is made. A query q is considered a match only if *all* of K LSH values match (a logical *AND* over the K hash functions). The decision for an element is then obtained by taking the *OR* over all partitions $\ell \in [L]$: the fuzzy PDS outputs membership if at least one fuzzy instance ℓ satisfies this per-partition AND condition. Such AND-OR semantics are commonly leveraged in the LSH literature to tune the false positive and false negative rates.

Proof. Fix any set Y inserted into the Fuzzy PDS, and any query point $q \in \mathcal{U}$. We proceed in two cases.

Case 1: Near case. Suppose there exists some $y^* \in Y$ such that $\mathsf{dist}(q, y^*) \le r_i$. For a fixed $\ell \in [L]$, success requires that
$$H_{\ell,k}(q) = H_{\ell,k}(y^*) \text{ for all } k \in [K].$$
Since $\mathsf{dist}(q, y^*) \le r_i$, by the definition of an LSH family (2), for each $k \in [K]$,
$$\Pr\big[H_{\ell,k}(q) = H_{\ell,k}(y^*)\big] \ge p_1.$$
By independence of the K hash evaluations, the probability that all $H_{\ell,k}$ collide is at least p_1^K.

Moreover, the LSH values are concatenated and subsequently hashed using a collision-resistant hash function H as
$$G_\ell(q) = H(H_{\ell,1}(q), \ldots, H_{\ell,K}(q)).$$
Thus, if $H_{\ell,k}(q) = H_{\ell,k}(y^*)$ for all $k \in [K]$, then, since the outer hash H is deterministic, $G_\ell(q) = G_\ell(y^*)$. Note that any additional collisions introduced by this final deterministic hash can only increase the probability of acceptance, so the above probability gives a valid lower bound on the final hashes colliding.

Now consider the L independent partitions. The probability that none of them succeeds is at most $(1 - p_1^K)^L$. Thus, with probability at least
$$1 - (1 - p_1^K)^L,$$
there exists $\ell \in [L]$ such that $G_\ell(q) = G_\ell(y^*)$.

Fix such an ℓ. During insertion, the value $G_\ell(y^*)$ was inserted into D. During query, the corresponding value $G_\ell(q)$ is checked against the same PDS value in D. Since these values are equal and the underlying PDS has false-negative probability 0, this lookup returns membership with probability 1. Since the check in the PDS succeeds, the final fuzzy PDS output is $b = 1$.

It follows that
$$\Pr[b = 1 \mid \exists y \in Y : \mathsf{dist}(q, y) \le r_i] \ge 1 - (1 - p_1^K)^L.$$
Thus, we may set
$$\rho_1 \ge 1 - (1 - p_1^K)^L.$$

Case 2: Far case. Now suppose that for all $y \in Y$, $\mathsf{dist}(q, y) \ge r_o$. For a fixed LSH instance $\ell \in [L]$ and a fixed element $y^* \in Y$, define
$$E_\ell := \bigwedge_{k\in[K]} \big(H_{\ell,k}(q) = H_{\ell,k}(y^*)\big).$$
That is, E_ℓ is the event that all K LSH values of q collide with those of y^* in partition ℓ. Since $\mathsf{dist}(q, y^*) \ge r_o$, by the definition of an LSH family (2), for each $k \in [K]$,
$$\Pr\big[H_{\ell,k}(q) = H_{\ell,k}(y^*)\big] \le p_2.$$
By independence of the K hash evaluations, we thus have $\Pr[E_\ell] \le p_2^K$. The value checked in the PDS, however, is the hash of the concatenation $G_\ell(\cdot)$.

For a fixed $y^* \in Y$, the event $G_\ell(q) = G_\ell(y^*)$ can occur either because (i) E_ℓ occurs, or because the outer hash H collides on two distinct LSH tuples. Let $p_H = \mathsf{negl}(\lambda)$ denote an upper bound on the outer hash's collision probability. Then
$$\Pr[G_\ell(q) = G_\ell(y^*)] \le p_2^K + \mathsf{negl}(\lambda).$$
Taking a union bound over all $y \in Y$, the probability that the derived fuzzy value $G_\ell(q)$ matches some value inserted into D is at most
$$\Pr\big[\exists y \in Y : G_\ell(q) = G_\ell(y)\big] \le |Y|(p_2^K + p_H).$$
Thus, for a fixed ℓ, the probability that the lookup succeeds because of a collision with some inserted fuzzy value is at most $|Y|(p_2^K + \mathsf{negl}(\lambda))$. Even if no such collision occurs, the underlying PDS may still return a false positive. Since the false-positive probability of Σ is at most p_{fpp}, the probability that repetition ℓ incorrectly returns membership is at most
$$|Y|(p_2^K + \mathsf{negl}(\lambda)) + p_{\mathsf{fpp}}.$$

Finally, the Fuzzy PDS combines the L LSH instances using OR logic, so the overall output is $b = 1$ if at least one repetition returns membership. Applying a union bound over all L instances gives

$$\Pr[b = 1 \mid \forall y \in Y : \mathsf{dist}(q, y) \geq r_o] \leq L\big(|Y|(p_2^K + \mathsf{negl}(\lambda)) + p_{\mathsf{fpp}}\big).$$

Thus, we may set

$$\rho_2 \leq L\big(|Y|p_2^K + p_{\mathsf{fpp}}\big) + \mathsf{negl}(\lambda).$$

By combining the two cases, the theorem follows. □

B ADDITIONAL CONTENT ON THE FUZZY PSI PROTOCOL

B.1 A Note on Spatial Hashing

While our protocol leverages LSH, which was developed in the context of plaintext nearest-neighbor search, one could instead use spatial hashing, which provides different correctness and efficiency guarantees. Spatial hashing was used by Garimella et al. [1] in the context of Fuzzy PSI to decrease the asymptotic complexity of checking whether a point $\mathbf{x} \in X$ is within distance δ of a point $\mathbf{y} \in Y$ under the L_∞ norm.

At a high level, the idea is to tile the input space U^d into equal-sized grid cells e.g., d-dimensional hypercubes of side length 2δ. Each cell is assigned a unique label. To avoid false negatives, for each $\mathbf{x} \in X$ the server inserts the labels of all cells that intersect the δ-ball around $\mathbf{x}$. These labels can then be packed into an oblivious key-value store (OKVS). The client, for each $\mathbf{y} \in Y$, computes the label of the cell containing $\mathbf{y}$ and queries the OKVS using standard PSI techniques. If $\|\mathbf{x} - \mathbf{y}\|_\infty \leq \delta$, then the cell containing $\mathbf{y}$ necessarily intersects the δ-ball around $\mathbf{x}$, and hence its label was inserted by the server. However, spatial hashing may still produce false positives: a cell that intersects the δ-ball around $\mathbf{x}$ may contain a client point $\mathbf{y}$ whose distance from $\mathbf{x}$ is greater than δ. Such candidates can be filtered by an additional exact distance check.

Since the encoding is exponential in d, this approach works best for low-dimensional spaces.

B.2 Correctness of our Fuzzy PSI Protocol

Theorem B.1 (Correctness). *Let Π_{FPSI} be the protocol in Fig. 4 and let $\lambda \in \mathbb{N}$ be the security parameter. Let Π_{PIR} be a correct PIR scheme and let Π_{OPRF} be a correct batch OPRF scheme, both with correctness error negligible in λ. Let Σ^* be an insertion-only PRF-wrapped Fuzzy PDS that is $(\rho_1, \rho_2, \delta_i, \delta_o)$-correct.*

At the end of executing $\Pi_{\mathsf{FPSI}}(X; Y)$, the server $\mathbf{S}$ outputs $|X|$ and the client $\mathbf{C}$ outputs a set $Z \subseteq X$ such that, for all $\mathbf{x} \in X$:

- *if there exists $\mathbf{y} \in Y$ such that $\mathsf{dist}(\mathbf{x}, \mathbf{y}) \leq \delta_i$, then*
$$\Pr[\mathbf{x} \in Z] \;\geq\; \rho_1 - \mathsf{negl}(\lambda),$$
- *if for all $\mathbf{y} \in Y$ we have $\mathsf{dist}(\mathbf{x}, \mathbf{y}) \geq \delta_o$, then*
$$\Pr[\mathbf{x} \in Z] \;\leq\; \rho_2 + \mathsf{negl}(\lambda).$$

Proof. Let Good be the event that all OPRF outputs obtained by the client are equal to the corresponding PRF outputs under $\mathsf{k}_{\mathsf{PRF}}$, and that all PIR queries are correct. Since Π_{OPRF} and Π_{PIR} are correct, except with negligible probability, and since the protocol invokes them only polynomially many times, we have $\Pr[\text{Good}] \geq 1 - \mathsf{negl}(\lambda)$.

Fix $\mathbf{x} \in X$, and let $b_{\mathbf{x}}$ be the membership bit computed by the client for $\mathbf{x}$. The client adds $\mathbf{x}$ to Z if and only if $b_{\mathbf{x}} = 1$.

Conditioned on Good, the computation performed by the client on input $\mathbf{x}$ is as follows (lines 12–18 in Fig. 4):

$$\begin{aligned} \mathbf{v} &\leftarrow \Sigma^*.\mathsf{GetFuzzyVals}(\mathbf{x}) \\ (\widetilde{Q}; \perp) &\leftarrow \Pi_{\mathsf{OPRF}}(\mathbf{v}; \mathsf{k}_{\mathsf{PRF}}) \\ (\mathbf{locs}, \mathbf{info}) &\leftarrow \Sigma^*.\mathsf{GetInfo}(\widetilde{Q}) \end{aligned}$$

Then for each $\mathsf{loc} \in \mathbf{locs}$ it runs

$$(\mathsf{st}_C, \mathsf{resp}_{\mathsf{loc}}; \perp) \leftarrow \Pi_{\mathsf{PIR}}.\mathsf{Query}((\mathsf{st}_C, \mathsf{loc}); (\mathsf{st}_S, D))$$

and appends resp to **resps**. Finally, the client checks:

$$b_{\mathbf{x}} \leftarrow \Sigma^*.\mathsf{CheckMemb}(\mathbf{resps}, \mathbf{info}).$$

Conditioned on Good, the OPRF output must satisfy $\widetilde{Q} = \{F_{\mathsf{k}_{\mathsf{PRF}}}(u)\}_{u \in \mathbf{v}}$. Moreover, each PIR response resp is exactly the value that would be obtained by querying location loc in the server's PDS database D. Hence, conditioned on Good, the vector **resps** is exactly the output of

$$\Sigma^*.\mathsf{Query}(D, \mathbf{locs}).$$

Therefore, conditioned on Good, the client's computation of $b_{\mathbf{x}}$ is equivalent to testing membership using the PRF-wrapped Fuzzy PDS, i.e., by executing the following:

$$\begin{aligned} \mathbf{v} &\leftarrow \Sigma^*.\mathsf{GetFuzzyVals}(\mathbf{x}), \\ (\mathbf{locs}, \mathbf{info}) &\leftarrow \Sigma^*.\mathsf{GetInfo}(\{F_{\mathsf{k}_{\mathsf{PRF}}}(u)\}_{u \in \mathbf{v}}), \\ \mathbf{resps} &\leftarrow \Sigma^*.\mathsf{Query}(D, \mathbf{locs}), \\ b_{\mathbf{x}} &\leftarrow \Sigma^*.\mathsf{CheckMemb}(\mathbf{resps}, \mathbf{info}). \end{aligned}$$

Thus the probability that $b_{\mathbf{x}} = 1$, conditioned on Good, is exactly the acceptance probability of the $(\rho_1, \rho_2, \delta_i, \delta_o)$-correct Σ^*.

We now consider the two cases.

Case 1: Assume that there exists $\mathbf{y} \in Y$ such that $\mathsf{dist}(\mathbf{x}, \mathbf{y}) \leq \delta_i$. By the correctness of Σ^*, conditioned on Good we have $\Pr[b_{\mathbf{x}} = 1 \mid \text{Good}] \geq \rho_1$. Therefore,

$$\begin{aligned} \Pr[\mathbf{x} \in Z] &= \Pr[b_{\mathbf{x}} = 1] \\ &\geq \Pr[b_{\mathbf{x}} = 1 \wedge \text{Good}] \\ &= \Pr[\text{Good}]\Pr[b_{\mathbf{x}} = 1 \mid \text{Good}] \\ &\geq (1 - \mathsf{negl}(\lambda))\rho_1 \\ &\geq \rho_1 - \mathsf{negl}(\lambda). \end{aligned}$$

Case 2: Assume that for all $\mathbf{y} \in Y$, $\mathsf{dist}(\mathbf{x}, \mathbf{y}) \geq \delta_o$. By the correctness of Σ^*, conditioned on Good we have that $\Pr[b_{\mathbf{x}} = 1 \mid \text{Good}] \leq \rho_2$. Thus we have that

$$\begin{aligned} \Pr[\mathbf{x} \in Z] &= \Pr[b_{\mathbf{x}} = 1] \\ &\leq \Pr[b_{\mathbf{x}} = 1 \mid \text{Good}]\Pr[\text{Good}] + \Pr[\neg\text{Good}] \\ &\leq \rho_2 + \mathsf{negl}(\lambda). \end{aligned}$$

Since this argument holds for every $\mathbf{x} \in X$, the theorem follows. □

B.3 Proof of Theorem 4.1

Proof. We proceed using a hybrid argument and describe a simulator Sim_S that simulates the server's view to show that

$$\left(\mathsf{View}_S^{\Pi}(X,Y), \mathsf{Out}_C^{\Pi}(X,Y)\right) \overset{c}{\approx} \left(\mathsf{Sim}_S(1^\lambda, Y, |X|), O_C\right).$$

Sim_S takes as input 1^λ, the set Y, and the set size n of the client, and runs the server's protocol honestly, including sampling $\mathsf{k}_{\mathsf{PRF}}$. It simulates the client messages as follows.

(1) **Simulate offline PIR phase.** In the PIR offline phase, it invokes the PIR setup simulator

$$\mathsf{Sim}_0^{\mathsf{PIR}}(1^\lambda, \mathsf{pp}, L\cdot m, L\cdot n).$$

to obtain the client and server PIR states st_C and st_S, respectively.

(2) **Simulate the OPRF.** Then, for each $i \in [|X|]$, it uses the server-view simulator for the OPRF scheme to simulate the corresponding batched OPRF execution. In particular, it invokes

$$\mathsf{Sim}_S^{\mathsf{OPRF}}(1^\lambda, n_{\mathsf{bSize}}, \mathsf{k}_{\mathsf{PRF}}),$$

where n_{bSize} is the batch size of the OPRF invocation.

(3) **Simulate online PIR phase.** Finally, to simulate the PIR queries, for each $i \in [|X|]$ and each $\ell \in [L]$, it invokes the PIR query simulator

$$\mathsf{Sim}_1^{\mathsf{PIR}}(\mathsf{st}_S),$$

to obtain the updated state st'_S and sets $\mathsf{st}_S \leftarrow \mathsf{st}'_S$.

We now consider the following sequence of hybrids.

Hybrid $\mathbf{Hyb}_0$: The real honest interaction.

Hybrid $\mathbf{Hyb}_1$: Identical to $\mathbf{Hyb}_0$, except instead of running the PIR setup protocol, the simulator Sim_S invokes the PIR setup simulator $\mathsf{Sim}_0^{\mathsf{PIR}}(1^\lambda, \mathsf{pp}, L\cdot m, L\cdot n)$. By security of the PIR scheme **??**, the view of the server in $\mathbf{Hyb}_1$ is identical to that in $\mathbf{Hyb}_0$ and therefore the two hybrids are indistinguishable.

Hybrid $\mathbf{Hyb}_{2,i}$ for $i \in [n]$: Same as $\mathbf{Hyb}_{2,i-1}$ (or in the case of $\mathbf{Hyb}_{2,1}$ it is the same as $\mathbf{Hyb}_1$) with the following change. The i-th OPRF interaction is replaced with the simulated view of $\mathsf{Sim}_S^{\mathsf{OPRF}}$. By OPRF privacy against the server (Def. 3), this is indistinguishable from the previous hybrid.

Hybrid $\mathbf{Hyb}_{3,i,\ell}$ for $(i,\ell) \in [n]\times[L]$. This hybrid is identical to the previous except that the ℓ-th PIR query for the i-th element in X is replaced by messages generated by the PIR query simulator $\mathsf{Sim}_1^{\mathsf{PIR}}$. By multi-query PIR index privacy (**??**), the server's view in $\mathbf{Hyb}_{3,i,\ell}$ is computationally indistinguishable from the previous hybrid.

Hybrid $\mathbf{Hyb}_{3,n,L}$ is exactly the distribution produced by Sim_S. Since $n \in \mathsf{poly}(\lambda)$ there are a polynomial number of hybrids in the sequence. Thus the view of the server in the simulated view and in the real execution are computationally indistinguishable and the theorem follows. □

B.4 Proof of Theorem 4.2

Proof. We prove this using a hybrid argument and define a simulator Sim_C that simulates the client's view. Sim_C takes as input the client's set X, the fuzzy intersection $Z \subseteq X$, set size $|Y|$, and the match pattern $\mathsf{match} : X \to \{0,1\}^L$. It runs the client's protocol honestly and simulates the server messages as follows.

(1) **Generate public parameters.** The simulator samples a dummy PRF key $\hat{k}$ and runs the setup algorithm honestly to generate public parameters pp^* and initialize the simulated data structure $\widehat{D}$:

$$(\widehat{D}, \mathsf{pp}^*) \leftarrow \Sigma^*.\mathsf{Setup}(\mathsf{sp}^*, F_{\hat{k}}, m).$$

(2) **Sample OPRF outputs.** In order to simulate the OPRF, the simulator maintains a random-function table Tab. For every $x \in X$, it computes

$$\mathbf{v}_x \leftarrow \Sigma^*.\mathsf{GetFuzzyVals}(x).$$

Then for every $v \in \mathbf{v}_x$, if $\mathsf{Tab}[v]$ is undefined, samples

$$\mathsf{Tab}[v] \leftarrow\!\$\ \{0,1\}^\lambda.$$

The simulated OPRF outputs for $x \in X$ are defined as

$$\widehat{Q}_x := \{\mathsf{Tab}[v]\}_{v\in\mathbf{v}_x}.$$

(3) **Program responses consistent with match pattern.** The simulator now programs $\widehat{D}$ to be consistent with the match pattern match. It does so by first computing the query locations and auxiliary information for every $x \in X$ as

$$(\mathbf{locs}_x, \mathbf{info}_x) \leftarrow \Sigma^*.\mathsf{GetInfo}(\widehat{Q}_x).$$

and then inserting them into the data structures as

$$\widehat{D} \leftarrow \Sigma^*.\mathsf{Insert}(\widehat{D}, \mathbf{locs}_x, \mathbf{info}_x)$$

Then for every (x,ℓ) such that $\mathsf{match}[x][\ell] = 0$, the simulator overwrites the corresponding location $\mathsf{loc}_{x,\ell} \in \mathbf{locs}_x[\ell]$ in $\widehat{D}$ with random rejecting values.
Finally, the remaining locations of $\widehat{D}$ are filled with random dummy entries.

(4) **Simulate the OPRF transcripts.** Instead of executing the OPRF protocol, the simulator invokes the client-view OPRF simulator on the client's OPRF inputs and the programmed outputs:

$$\mathsf{Sim}_C^{\mathsf{OPRF}}(1^\lambda, \{\mathbf{v}_x\}_{x\in X}, \{\widehat{Q}_x\}_{x\in X}).$$

(5) **Simulate the PIR interaction.** When the client issues PIR queries for the locations in $\mathbf{locs}_x$, the simulator answers using the simulated database $\widehat{D}$, or invokes the client-view PIR simulator if PIR is modeled ideally. The resulting reconstructed responses are exactly those programmed above, and therefore the client obtains the match pattern match.

We now consider the following sequence of hybrids.

Hybrid $\mathbf{Hyb}_0$: The real honest protocol interaction.

Hybrid $\mathbf{Hyb}_1$: Same as $\mathbf{Hyb}_0$, except instead of using the output of the PRF $F_{\mathsf{k}_{\mathsf{PRF}}}$ evaluated on the client's OPRF inputs replace them with a random function. By the pseudorandomness property of the PRF, $\mathbf{Hyb}_0$ and $\mathbf{Hyb}_1$ are computationally indistinguishable.

Hybrid $\mathbf{Hyb}_2$. Same as $\mathbf{Hyb}_1$, except replace the real OPRF transcripts with transcripts generated by the client-view OPRF simulator $\mathsf{Sim}_C^{\mathsf{OPRF}}$, using the same client inputs and the same randomly sampled outputs used to replace the PRF evaluations. By OPRF security against a semi-honest client (Def. 3), $\mathbf{Hyb}_1$ and $\mathbf{Hyb}_2$ are computationally indistinguishable.

Hybrid $\mathbf{Hyb}_3$. Same as $\mathbf{Hyb}_2$, except replace the real fuzzy PDS with the programmed simulated database $\widehat{D}$, which agrees with the real execution on the responses specified by match and contains random dummy values elsewhere. By the security of the PRF wrapping and the client-view of the PIR interface, the client's view in $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_3$ are computationally indistinguishable.

Observe that the hybrid $\mathbf{Hyb}_3$ is exactly the distribution produced by $\mathsf{Sim}_{\mathsf{C}}$. Since there are a constant number of hybrids in the sequence, the view of the client in the simulated view and in the real execution are computationally indistinguishable and the theorem follows. □

REFERENCES

[1] Gayathri Garimella, Mike Rosulek, and Jaspal Singh. 2022. Structure-Aware Private Set Intersection, with Applications to Fuzzy Matching. In *CRYPTO 2022, Part I (LNCS)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.), Vol. 13507. Springer, Cham, 323–352. https://doi.org/10.1007/978-3-031-15802-5_12